
Trabalho Prático de Avaliação #2 - Comunicação e Sincronização entre Processos

(25% da nota final)

Objetivo Geral

Desenvolver uma aplicação que simula o funcionamento de um sistema de embarque de passageiros num aeroporto, utilizando processos concorrentes e comunicação via memória partilhada. O objetivo é gerir o fluxo de passageiros para o embarque, levando em consideração a prioridade dos passageiros, a disponibilidade de recursos (como portões e agentes de embarque) e o tempo de espera.

Descrição do Sistema

O sistema será dividido em dois componentes principais:

1. Aeroporto (Servidor)

O aeroporto é o "processo servidor", que gere todos os recursos e a comunicação com os passageiros. As suas funções principais serão:

- Triagem de passageiros: O servidor deve receber a chegada de passageiros, atribuindo-lhes uma prioridade com base em informações como a classe do bilhete ou o tempo de chegada.
- Gestão da fila de embarque: O servidor mantém uma fila de passageiros, organizada por prioridade (alta, média, baixa).
- Atribuição de portões e agentes de embarque: Quando o embarque de um passageiro é autorizado, o servidor deve alocar um portão disponível e um agente de embarque para o passageiro.
- Registo de operações: O servidor mantém um registo (log) com a hora de chegada dos passageiros, a sua prioridade, o tempo de espera e a duração do embarque.

2. Passageiros (Clientes)

Cada passageiro será um processo cliente, que possui:

- Chegada ao aeroporto: Os passageiros chegam ao aeroporto de maneira aleatória, a partir de tempos definidos pelo sistema.
- Prioridade: Cada passageiro terá uma prioridade associada, que pode ser gerada aleatoriamente ou definida com base em critérios como a classe do bilhete (primeira classe, classe executiva ou económica) ou o tempo de chegada.
- Aguardando embarque: O passageiro espera até que a sua vez chegue na fila de embarque, com base na prioridade definida.

Requisitos Técnicos

A implementação deverá ser feita utilizando Python, com foco nos seguintes pontos:

- Memória partilhada: Para permitir que os processos (servidor e clientes) partilhem dados de forma eficiente, deverá ser usada memória partilhada, onde o servidor e os passageiros podem ler e escrever dados, como a fila de embarque e o estado dos portões.
- Semáforos e sincronização: Para garantir que o sistema funcione sem condições de corrida

(onde múltiplos processos tentam aceder a recursos ao mesmo tempo, resultando em falhas), o uso de semáforos e outras técnicas de sincronização é obrigatório.

- Processos ou threads: Cada passageiro deve ser representado por um processo ou thread, que simula a chegada do passageiro ao aeroporto e a sua espera até o embarque. O servidor também deverá ser um processo, que gere a fila de embarque e as alocações de recursos.

Regras e Funcionalidades Mínimas

Aqui estão as funcionalidades devem ser implementadas:

1. Prioridade de embarque:
 - Passageiros com prioridade alta (como passageiros da primeira classe ou passageiros com embarque urgente) devem ser atendidos primeiro.
 - Passageiros com prioridade média ou baixa serão atendidos conforme a ordem da fila, com base na sua chegada ao aeroporto e na sua prioridade.
 - Passageiros de prioridade alta podem interromper o embarque de outros passageiros (caso a implementação permita essa funcionalidade de interrupção).
2. Portões e Agentes de Embarque:
 - Cada portão de embarque só pode ser utilizado por um passageiro de cada vez.
 - Agentes de embarque (simulados por processos) devem ser alocados para acompanhar o processo de embarque de cada passageiro.
3. Registos e Monitorização:
 - O sistema deverá registar:
 - Hora de chegada dos passageiros ao aeroporto.
 - Nível de prioridade de cada passageiro.
 - Tempo de espera do passageiro na fila até ser atendido.
 - Duração do embarque (quanto tempo leva para o passageiro ser embarcado).
 - Hora de embarque do passageiro.
4. Cenários de Alta Demanda:
 - Em algumas simulações, o sistema deverá ser capaz de simular períodos de alta procura, onde muitos passageiros chegam ao aeroporto ao mesmo tempo, aumentando a complexidade da gestão da fila de embarque.
5. Desistências:
 - Se um passageiro esperar por muito tempo na fila de embarque (por exemplo, mais do que um limite de tempo definido), ele pode "desistir" de embarcar. O sistema deve registar esta desistência e remover o passageiro da fila.

Outras Funcionalidades Opcionais

Aqui estão algumas funcionalidades extras que podem ser implementadas como melhorias:

1. Tempo de embarque variável: O tempo de embarque pode ser ajustado conforme a prioridade do passageiro. Passageiros de prioridade alta podem ter um tempo de embarque mais rápido, enquanto passageiros de baixa prioridade podem demorar mais.
2. Suporte a múltiplos portões de embarque: Se o aeroporto tiver mais de um portão disponível, o sistema deverá distribuir os passageiros entre os portões de maneira eficiente, garantindo

que um portão não fique sobrecarregado enquanto outro está ocioso.

3. Simulação de surtos: Durante o dia, podem ocorrer períodos com um grande número de passageiros tentando embarcar ao mesmo tempo. O sistema deve gerir essas "emergências" sem falhas.
4. Visualização gráfica ou em consola: O sistema pode exibir a fila de embarque em tempo real, seja numa interface gráfica ou através de uma representação em texto na consola, mostrando como a fila está a ser organizada e os passageiros a serem atendidos.
5. Gestão de desistências: Se um passageiro desistir de embarcar após muito tempo de espera, o sistema deverá registar-o como "desistido" e mostrar um resumo das desistências no log.

Entrega Esperada

1. Código-fonte completo: O código deverá ser bem organizado, modular e comentado, para que seja fácil entender a implementação.
2. Arquivos de log: O sistema deverá gerar automaticamente um arquivo de log contendo todas as operações realizadas, como a chegada de passageiros, alocação de portões e agentes, e desistências.
3. Relatório:
 - Explicação da arquitetura da solução: Os alunos devem descrever como organizaram o sistema, quais processos foram criados e como eles se comunicam.
 - Decisões tomadas e dificuldades: Devem explicar as escolhas feitas durante a implementação e quaisquer dificuldades encontradas.
 - Propostas de melhorias futuras: Sugestões para aprimorar o sistema, como a adição de novas funcionalidades ou otimizações.

Memória partilhada ou passagem de mensagens?

Deixar socket sempre ativo e só quando houver a última mensagem é que se fecha o socket.